

Operating Systems as Code: The Missing Layer in Cloud Native Platforms

Provisioned, not installed, with Flatcar Container Linux

Jane Doe

whoami



Jane Doe

Flatcar Maintainer

Software Engineer

@janedoe

Agenda

- The community behind Flatcar
- What problem do we solve?
- Operating Systems as Code
- Provisioned, not installed
- Immutable by design & A/B updates
- Release channels
- Live demo

Community Stewarded

What that means, and who we are

Community stewarded

- **Fully open, fully free:** Apache 2.0, public roadmap, public issue tracker
- **No single owner:** the project and trademark belong to the CNCF and the Linux Foundation
- **CNCF project:** neutral governance under the Cloud Native Computing Foundation
- **You can join:** SIG meetings are public; anyone can propose changes or maintain

Flatcar is a **CNCF** project



CLOUD NATIVE
COMPUTING FOUNDATION

The community

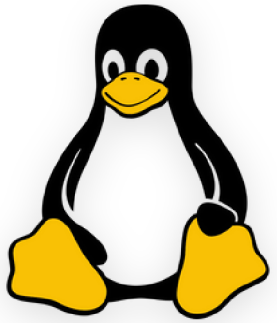


We work with

- **Gentoo** and **Fedora CoreOS**
- **systemd**, **Dracut**, **Grub**, **Afterburn**
- Openwall **oss-security** non-disclosure list
- Co-founders of the **UAPI Group**, cross-distro SIG for image-based Linux









Runs everywhere



In production at



What Problem Do We Solve?

Why the OS is still the odd one out

Everything is code. Almost.

Already declarative

- Dockerfiles
- Kubernetes manifests, Helm charts
- GitOps workflows
- CI/CD pipelines

Still treated differently

- Host OS installed interactively
- Changed over time
- Repaired manually when it breaks

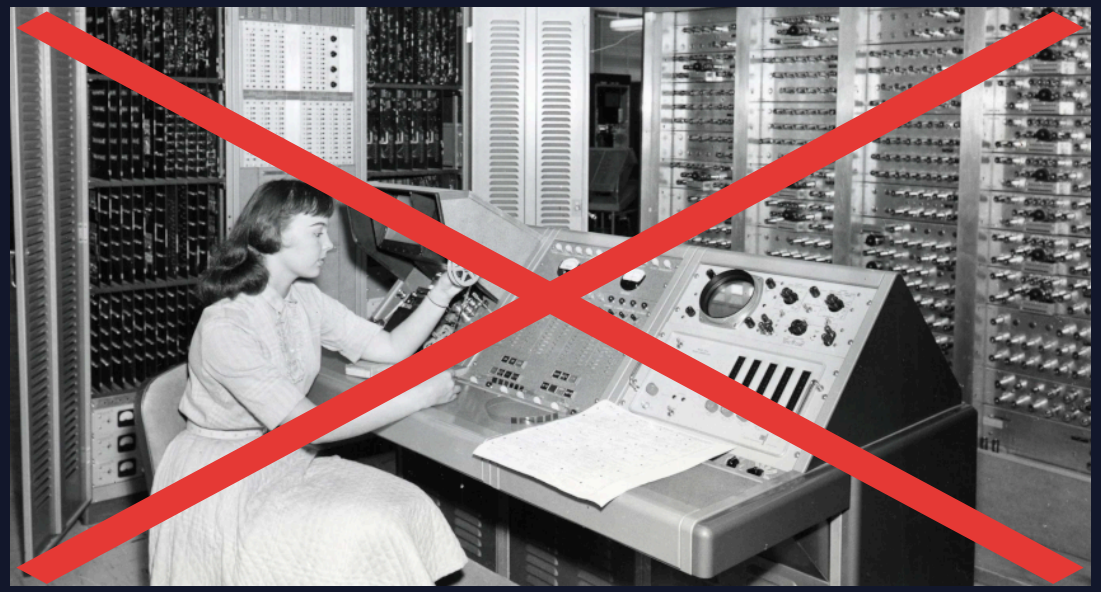
Flatcar?



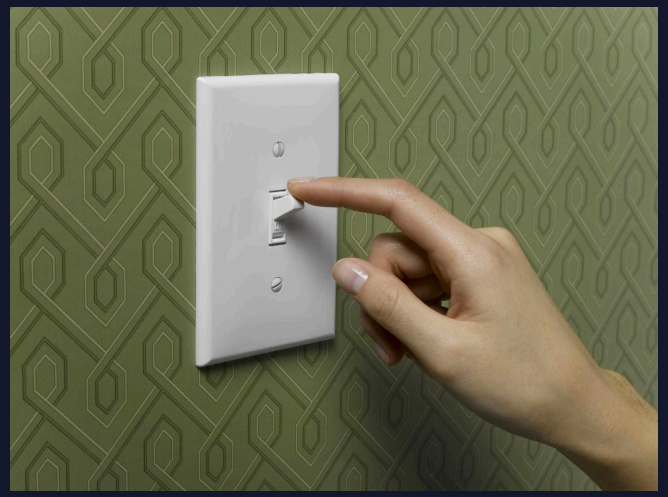
Flatcar!


RINGLING BROS. THE GREATEST SHOW ON EARTH BARNUM & BAILEY
RINGLING BROS. THE GREATEST SHOW ON EARTH BARNUM & BAILEY
RBBX 80701

The UX we chose



General-purpose Linux



Flatcar

Operating Systems as Code

A versioned, declarative artifact. Provisioned, not installed.

Provisioned, not Installed

Traditional install

- Interactive choices during setup
- Every machine ends up a little different
- Manual updates, package churn, drift
- No separation between OS and config

Flatcar: provisioned

- One declarative config, applied at boot
- Every machine identical, reproducible
- Whole-OS atomic updates, no drift
- Same idea as containers, with the config in Git

Flatcar Workflow



The Container Philosophy, for the OS

	Containers	Operating System
Recipe	Dockerfile	Butane / Ignition
Image	Docker image versioned, immutable	Flatcar image versioned, immutable
Instance	Container disposable	Node reprovisionable
Update	New image tag replace, don't patch	New OS release A/B swap, don't patch

What Butane looks like

```
# Butane config header: declares this is a Flatcar config
variant: flatcar
version: 1.0.0

# Drop in a systemd unit. Flatcar installs and starts it at first boot
systemd:
  units:
    - name: nginx.service
      enabled: true
      contents: |
        [Service]
        ExecStart=/usr/bin/docker run --rm -p 80:80 nginx
        [Install]
        WantedBy=multi-user.target
```

Boot it in a VM

```
# Transpile Butane to an ignition config (JSON)
$ butane -o config.ign config.bu

# Boot Flatcar in QEMU with the config applied
$ ./flatcar_production_qemu.sh -i config.ign
```

The config gets handed to Flatcar as **user-data**. Same channel every major cloud, hypervisor, and bare-metal provisioner uses to seed an instance.

It came up serving

```
$ systemctl is-active nginx.service
active

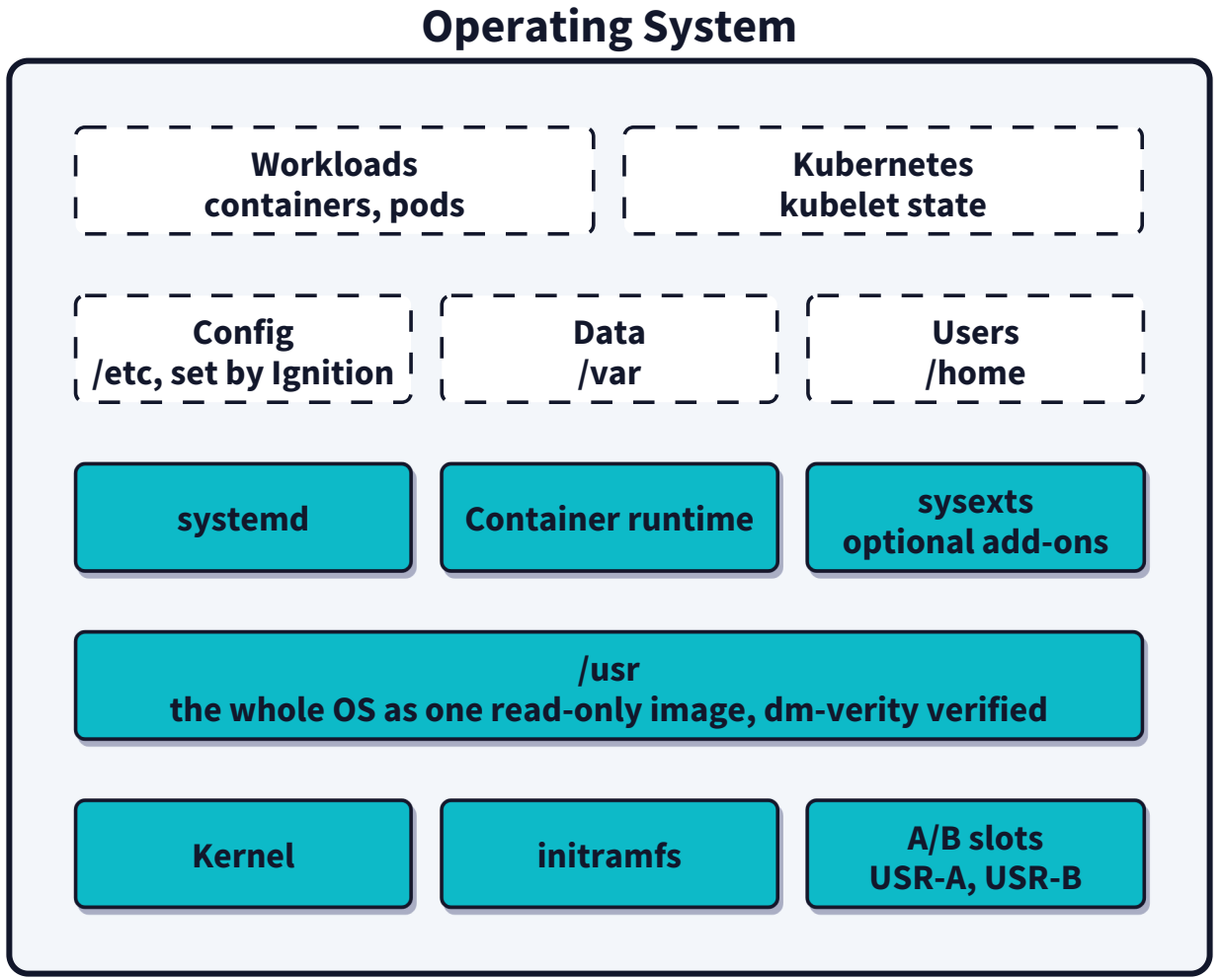
$ curl -sI http://localhost
HTTP/1.1 200 OK
Server: nginx/1.27.3
Content-Type: text/html
```

No login. No post-boot setup. The machine came up already doing its job.

Immutable by Design

- **First boot** provisions from config. After that: the base OS doesn't change.
- The OS is **read-only** and **dm-verity protected**.
- **No package updates**: the entire OS updates as one unit.
- Same config + same base image = **identical machine every time**.
- **sysexts** for optional add-ons.

What is Immutable?



Writable: yours

Read-only: Flatcar's

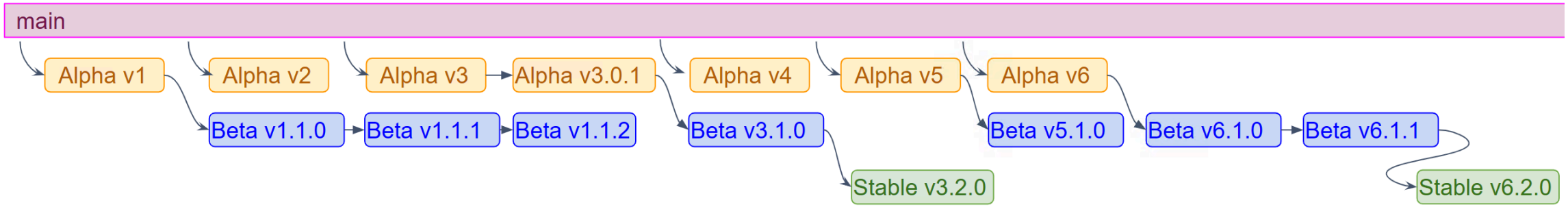
A/B Updates

1	**Fresh install**	A 1.2.3	B 1.2.3	Image ships both slots identical. Only true at first boot.
2	**Update staged**	A 1.2.3	B 1.2.4	New image written over the passive slot. No downtime.
3	**Reboot swaps**	A 1.2.3	B 1.2.4	Boot flips to B. If it fails, reboot back to A at 1.2.3.
4	**Steady running**	A 1.2.3	B 1.2.4	B live. A untouched: it's your rollback target.
5	**Next update**	A 1.2.5	B 1.2.4	1.2.5 overwrites the old 1.2.3 on A. Next reboot: A active, B rollback.



↺ The passive slot is **never auto-synced**. It's your rollback until the next update lands.

Channels



- **Alpha**: fully tested, may have incomplete features. Developers.
- **Beta**: production-ready. Canary alongside stable.
- **Stable**: widespread production. Promoted from beta.
- **LTS**: slower-moving track for environments that need it.

> Every commit runs the full test suite: unit, integration, and product tests across every supported cloud. By the time a version ships, even **Alpha**, it's been through hundreds of runs.

Demos!

github.com/flatcar/flatcar-demos

Thank you!



Join our Discord!

flatcar.org

github.com/flatcar

discord.gg/PMYjFUsJyq

> **Office hours** · every 2nd Thu, 16:30 CEST **Dev sync** · every 4th Thu, 16:30 CEST

Everyone welcome. Users, contributors, or just curious. Ask questions, get help, share what you're building!